03/07/2025 - 28/07/2025

For Ground Truth

1. Modify the code to save the mesh for each frame
2. Do ray tracing to get ground truth for each pixel of each camera for each frames
3. Save the triangle location with distance to vertices OR uv maps location

For Constructing the Point Cloud

4. Transform the depth maps into point cloud
5. Solve the extrinsic calibration and apply it to the point cloud to merge everything

1. Saving as .ply
https://docs.blender.org/api/current/bpy.ops.wm.html#bpy.ops.wm.stl_export

2. Ray Tracing options:

- https://pypi.org/project/raytracing/
- https://docs.pyvista.org/examples/01-filter/poly_ray_trace.html - might be useful
  https://github.com/nv-tlabs/3dgrut - more complex one - might be good for time-changing ones?
- https://towardsdatascience.com/python-libraries-for-mesh-and-point-cloud-visualization-part-1-daa2af36de30/ - good guide

- https://www.open3d.org/docs/release/tutorial/geometry/ray_casting.html
- https://blog.michelanders.nl/2018/05/raytracing-concepts-and-code.html
- https://www.open3d.org/
- https://www.open3d.org/docs/release/tutorial/geometry/mesh.html

Maybe a good option for working with the intel realsense depth camera in the future:
https://www.open3d.org/docs/release/tutorial/sensor/realsense.html

3. Saving ground truth for each frame
https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-generating-camera-rays/generating-camera-rays.html
https://antongerdelan.net/opengl/raycasting.html

- Need to keep in mind the distances between the cameras and the origin camera. All the cameras from one origin point

Why Save This Info?

- triangle_id allows mapping to mesh structure.

- hit_xyz is useful to generate a point cloud directly.
- dv1–3 gives you geometric context (e.g., useful for mesh-aware learning or barycentric reconstruction).
- pixel_x, pixel_y is useful for reverse projection if you want to map image-space data back to 3D space.

- What is a .JSON file?
- https://www.geeksforgeeks.org/python/reading-and-writing-json-to-a-file-in-python/

Camera information from blender:

```
def point_camera_to_origin(camera_obj, target=(0, 0, 1)):
    target_vec = mathutils.Vector(target)
    direction = target_vec - camera_obj.location
    rot_quat = direction.to_track_quat('-Z', 'Y')  # '-Z' is camera's forward axis
    camera_obj.rotation_euler = rot_quat.to_euler()

def add_cameras():
    distance = 5  # Distance from center
    camera_positions = [
        (distance, 0, 2),   # Right
        (-distance, 0, 2),  # Left
        (0, distance, 2),   # Front
        (0, -distance, 2)   # Back
    ]

    cameras = []

    for i, pos in enumerate(camera_positions):
        cam_data = bpy.data.cameras.new(name=f"Camera_{i+1}")
        cam_obj = bpy.data.objects.new(name=f"Camera_{i+1}", object_data=cam_data)
        bpy.context.collection.objects.link(cam_obj)

        cam_obj.location = pos
        point_camera_to_origin(cam_obj)

        cameras.append(cam_obj)

    return cameras
```

Focal length: 50

Resolution: 1920px x 1080px

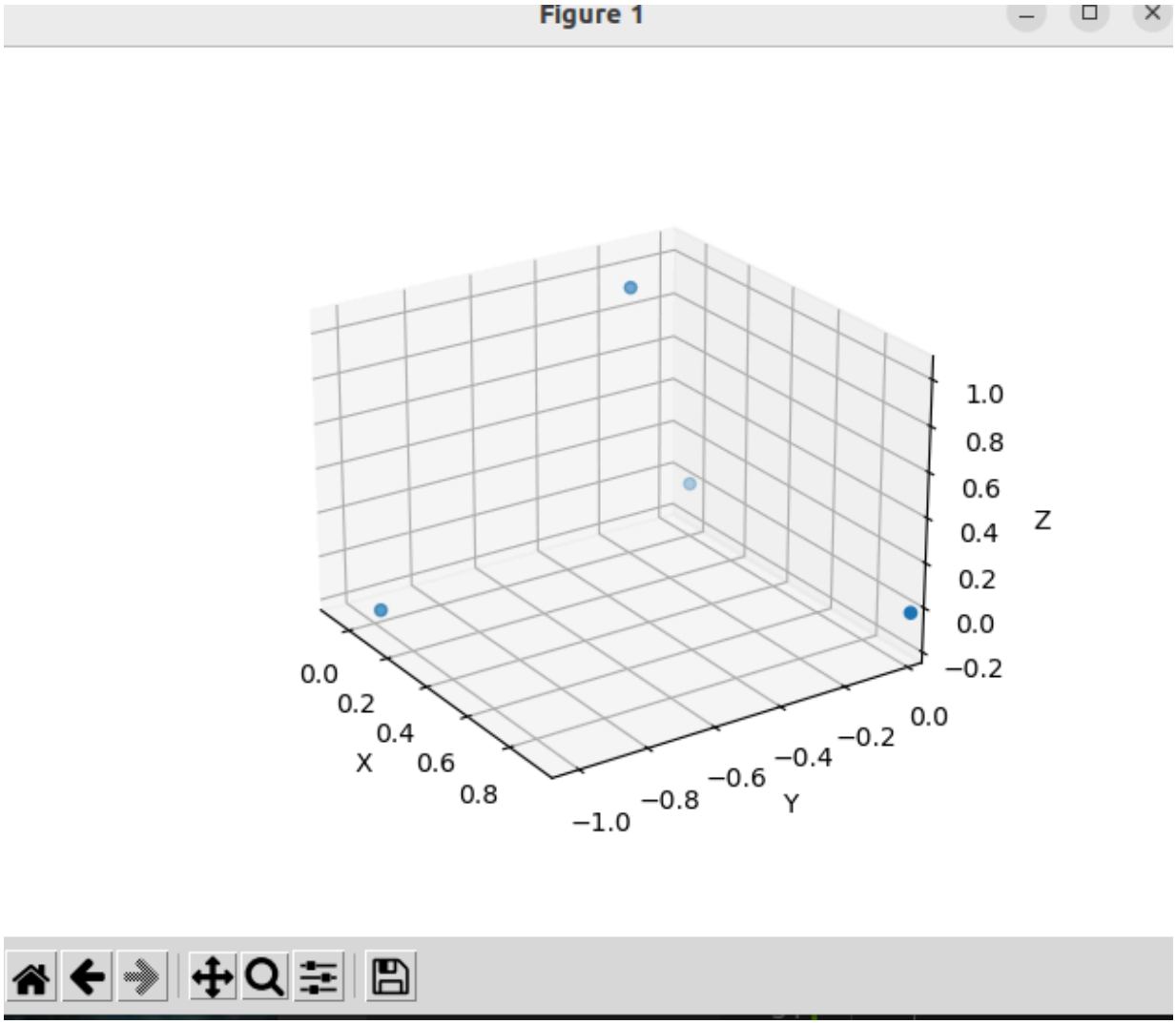
Frame Rate: 30fps

Notes:

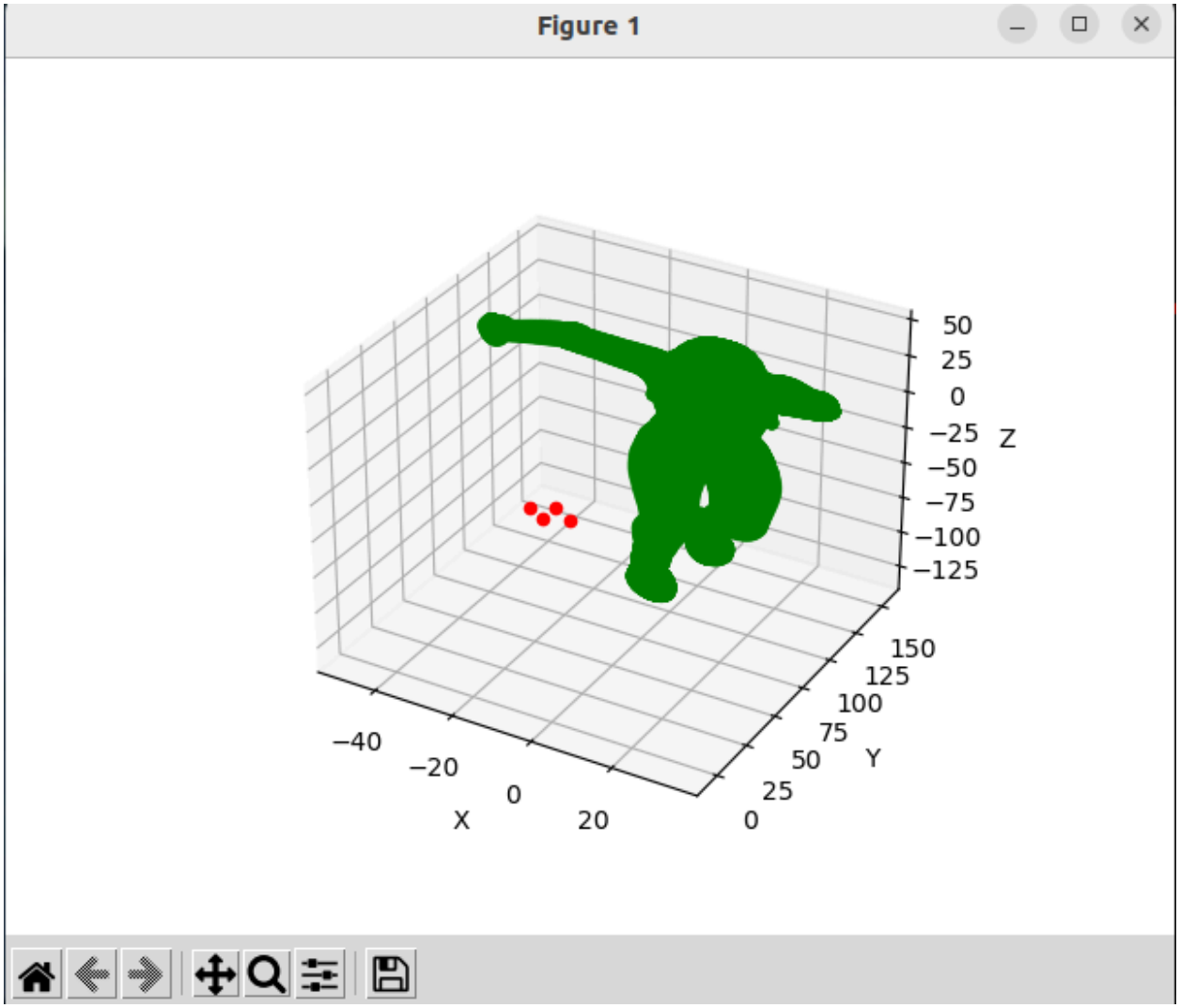
- These cameras would be facing the origin which is where the mesh needs to be placed, so I need to make sure I set up the scene in that way.
- First thing is to set up the .JSON file for saving the data.

Blender Python API Documentation:
https://docs.blender.org/api/current/bpy.ops.wm.html#bpy.ops.wm.obj_export

Inversed cameras:



With the character mesh:



- I thought I could simply import the models and cameras as they were for ray_casting in open3d but I guess that's not the case
- There is also currently an issue with the .json file that is being created, I've noticed that the same face shows up over multiple pixels, I hope that won't be a problem later when I try to read and combine the data.
    - I can try setting up a sample code for putting together the pointcloud from the .json file and the depth camera data if I am completely stuck on this for now. But I'm hesitant to mauve on and have even more unfinished work before I complete this one. But I'm just completely stuck here, what do I do now, how am I going to finish this one time. Its not even like I started it late, I've been working on this for weeks and I still can't figure it out! Should I really just wait for my supervisor to get back and discuss with him? I just feel so guilty about not being able to do it, maybe I should just work on putting together the point clouds so that I at least have something to show him.

- Is the reason the camera thing is not working because of calibration?
    - In general I need to learn to calibrate a depth camera properly
    - Also maybe for the intrinsic parameters I should use a .xml file instead of a jason file? I just don't know how to set it up.
    - https://www.youtube.com/watch?v=teHGdlGhQZo

https://python.land/virtual-environments/virtualenv#Python_venv_activation

https://www.youtube.com/watch?v=pk77M4KBg2Q

https://www.open3d.org/docs/latest/tutorial/Basic/rgbd_image.html

Saving the camera extrinsics:

- https://github.com/isl-org/Open3D/discussions/3836
- https://devtalk.blender.org/t/getting-different-extrinsic-matrix-values-when-running-blender-through-the-terminal-versus-when-running-using-blender-provided-console/30200
- https://polyscope.run/py/

Capstone Progress Update - 06/08/25

Tasks: Data Collection and Ground Truth Generation from the Blender Simulation

For Ground Truth

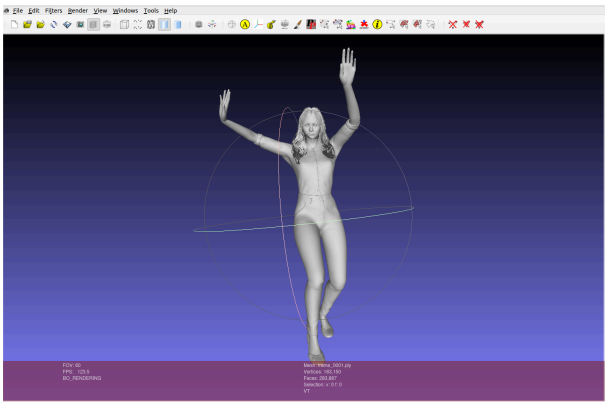
1. Modify the code to save the mesh for each frame
2. Do ray tracing to get ground truth for each pixel of each camera for each frames
3. Save the triangle location with distance to vertices OR uv maps location

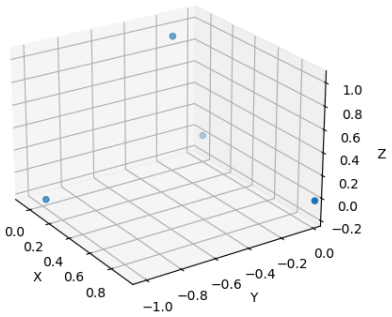
For Constructing the Point Cloud

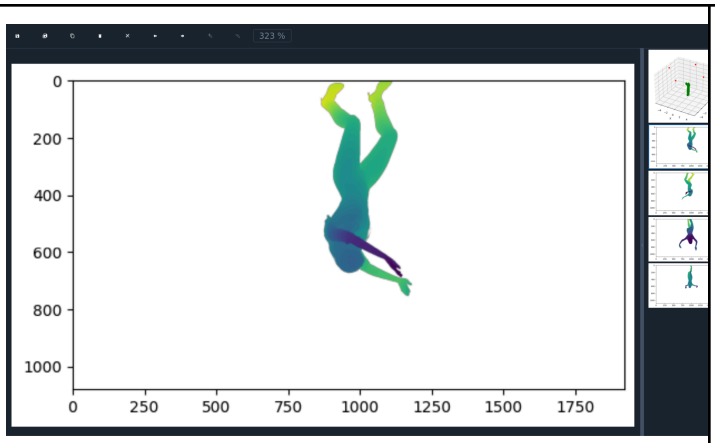
4. Transform the depth maps into point cloud
5. Solve the extrinsic calibration and apply it to the point cloud to merge everything

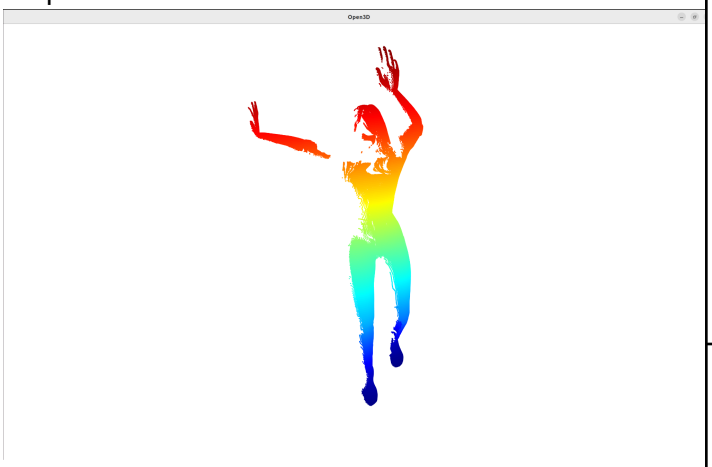
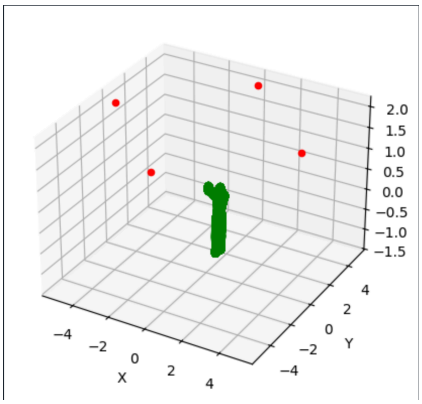
---

Task	Status	Details	Challenges
4		File: **model_save.py** Combined all the separate meshes under the armature into a single mesh then looped through each frame and saved the selected mesh using: *bpy.ops.wm.ply_export*(filepath=filepath,	Getting used to Blender and its python API, as it was difficult to find relevant resources and practical tutorials. The sample scripts provided by my supervisor, the blender py api and open3d documentation, ChatGPT and Gemini were the most useful resources overall.

		export_selected_objects=True,
		#export_uv=True,
		export_normals=True,
		export_triangulated_mesh =True,
		)
		
		 
		
		Notes:
		● Needed to scale the mesh down to 0.01 to fit the Open3D scene.
		● I'm not sure if I needed to export the uvs as well but now that I have the code set up I can always do that again later if required.
			Resources:
			https://docs.blender.org/api/current/bpy.ops.wm.html#bpy.ops.wm.obj_export

2		File: **ray_tracing.py** Each model was imported frame by frame. The ray tracing was done using the Open3D *cast_rays* function. The camera intrinsic and extrinsic matrices were saved from blender into a *camera_data.json* file to be used to create: *geometry.RaycastingScene.create_rays_pinhole* From the example in the documentation, the *cast_rays* function returns a dictionary including the triangle index of the triangle that was intersected and the distance to the intersection that can be used to visualise the output from each camera.	The biggest challenge was setting up the scene in Open3D to match the Blender scene. ● The coordinate systems are different between Blender and Open3D causing positioning problems  Another issue was that even after getting the correct camera extrinsic matrices in blender, the rays were not aimed at the model, the positioning is still not perfectly accurate to the blender scene.

<table>
<tr>
<td></td>
<td></td>
<td> 

Output from one camera:
 
</td>
<td> 

The mats had to be rotated to point at the model using its center point as the target. Though this worked, I’m worried it's not accurate to the blender scene in which the cameras are always looking at the world origin, not following the model.
From chatGPT: Blender stores transform changes (location/rotation) but doesn’t apply them to <code>matrix_world</code> until the scene is updated.

Resources:
https://www.open3d.org/docs/0.14.1/tutorial/geometry/ray_casting.html
https://github.com/isl-org/Open3D/issues/6508</td>
</tr>
<tr>
<td>3</td>
<td></td>
<td>Files: ray_tracing.py and plot_fromjson.py

The output from cast_rays can be used to obtain:

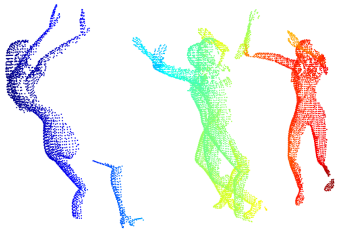
camera_hit_list.append({
"pixel": [x, y],
"triangle_id": int(tri_id),
"hit": hit_point.tolist(),
"dv1": dv1,
"dv2": dv2,
"dv3": dv3
})

This is saved into a .json file for each frame from each camera. The file structure is the same as the renders saved from blender.

Used plot_fromjson.py To test if the ground truth was saved properly, easy to access and useable I retrieved, plotted and combined the hit data from</td>
<td>Time and storage space.

There are a large amount of ground_truth files and renders so I needed to move them to an external hard drive.
2 frames from each camera for all the data are saved to the github for testing.

Aligning the point clouds was difficult to figure out.

  </td>
</tr>
</table>

		each camera's *.json* file for one frame into a pointcloud:  Note: ● I'm pretty proud of this. It took quite a while to figure out. Even the model looks like she is celebrating :) ● I chose a *.json* file because the output from the ray casting was already a dictionary and the structure made the data easy to open and read, though it was a bit bigger than storing a *.txt* file.	As a work around I plotted each cloud at the center point of the cloud from the first camera. Resources: https://www.geeksforgeeks.org/python/reading-and-writing-json-to-a-file-in-python/
4		File: **render_to_mesh** Using the depth and rgb images to create an rgbd image and converting it to a point cloud with the camera intrinsics saved in the *camera_data.json* file.	Just plotting the point clouds from the depth images also included the background:

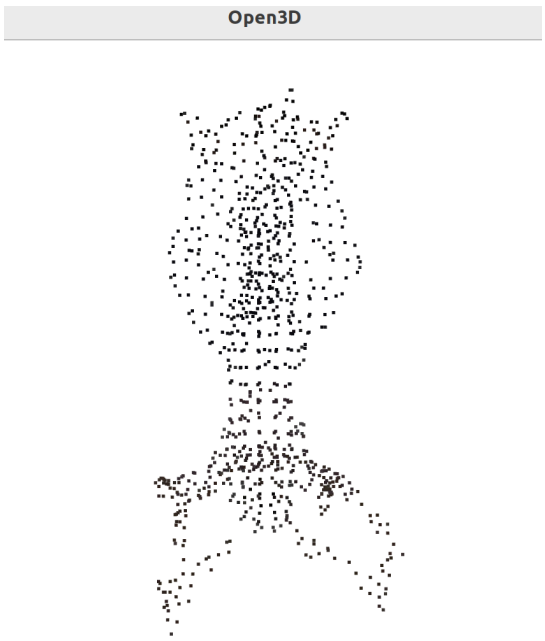
<table>
<tr>
<td></td>
<td></td>
<td> 

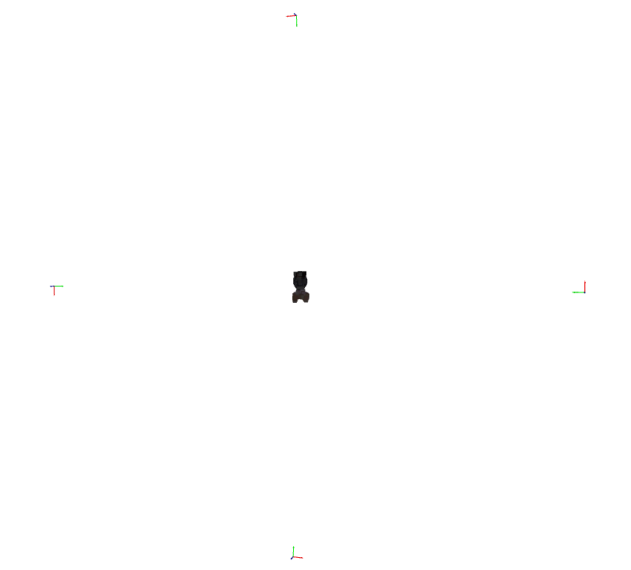
Note:
● Need to obtain the max distance from blender again to check and normalise the depth images</td>
<td> 

I needed to normalise the depth image with opencv.
● I used ChatGPT and it used 3m as the max depth, I’m not sure if there is a specific parameter for the cameras in blender but in the scene the cameras are placed 5m away from the origin so I changed the max_depth to 5m instead

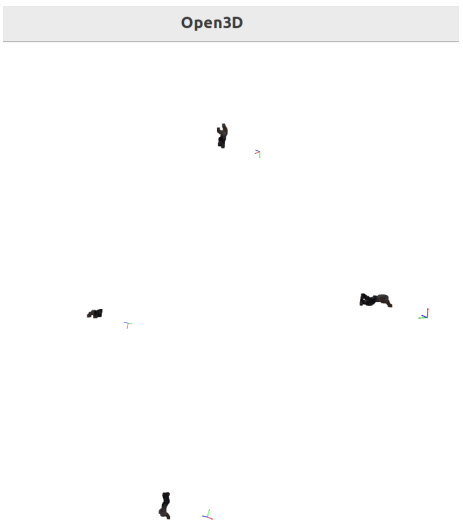
Resources:
https://www.open3d.org/docs/0.7.0/python_api/open3d.geometry.create_rgbd_image_from_color_and_depth.html
https://numpy.org/doc/2.1/reference/generated/numpy.ndarray.ndim.html</td>
</tr>
<tr>
<td>5</td>
<td></td>
<td>This is original downsampled without transforming with the extrinsics:</td>
<td>I can transform the pointcloud with the camera extrinsics but the results are not correct so I suppose the previous output was more correct.

This is with transforming to the extrinsics:</td>
</tr>
</table>





With camera coordinate systems^.



- I think the rotations are at least correct now
- I think the problem is that I need to transform the extrinsic matrices to a single world origin point?
- Or maybe I just need to rotate the mesh to match camera views?
- Or maybe I didn't save the extrinsics properly originally?
- Have I just misunderstood how to use the extrinsics?

I also tried to transform them to the center of the original cloud like the ground truth one but that didn't work either.

Resources:
https://www.open3d.org/docs/release/python_example/camera/index.html
https://www.open3d.org/html/tutorial/geometry/transformation.html

I want to properly learn about camera transforms.

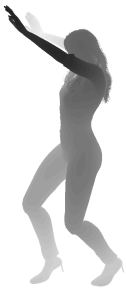
Capstone Progress Update - 28/08/25 - Completed Ray Tracing the Depth Render to Point Cloud Conversion

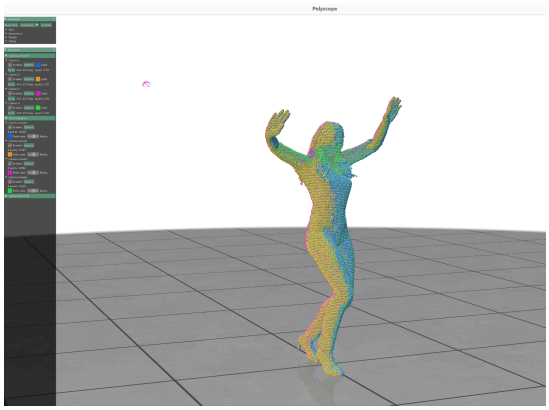
- Current Issue - Scaling challenges.

Scale Comparisons

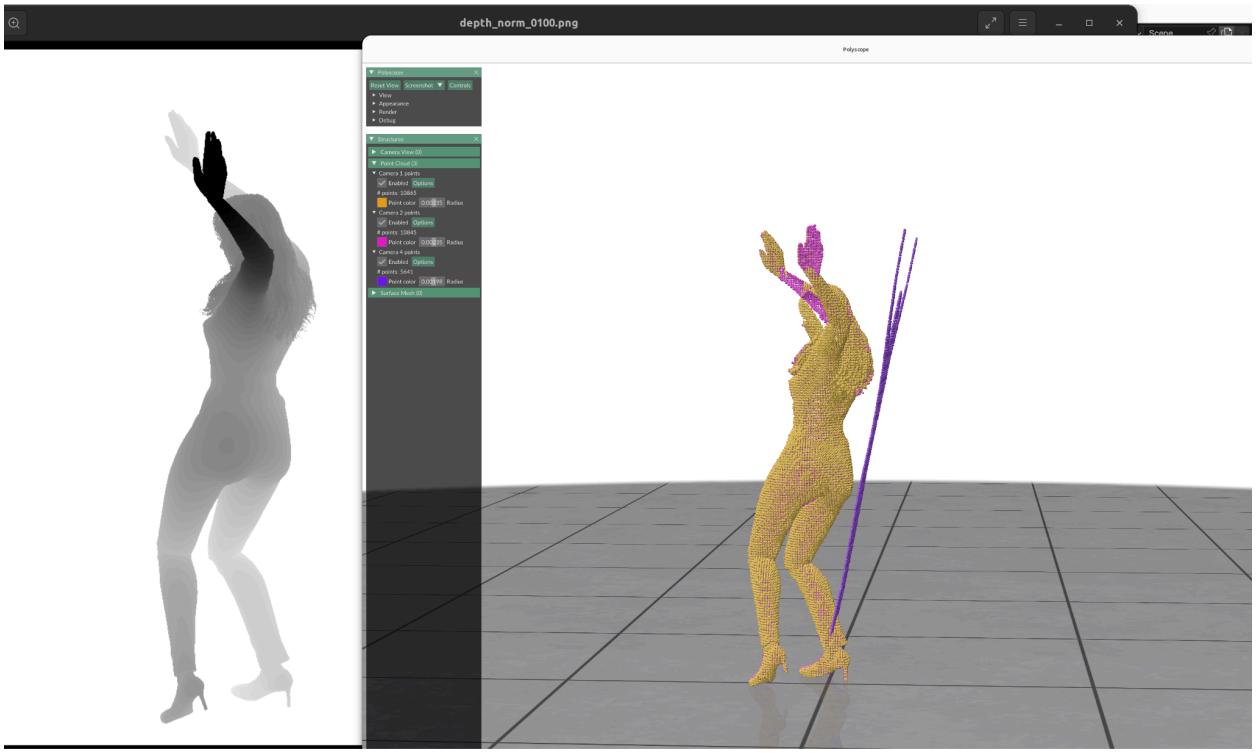
Originally selected scale: 4.5 - 5.5

Frame 1:



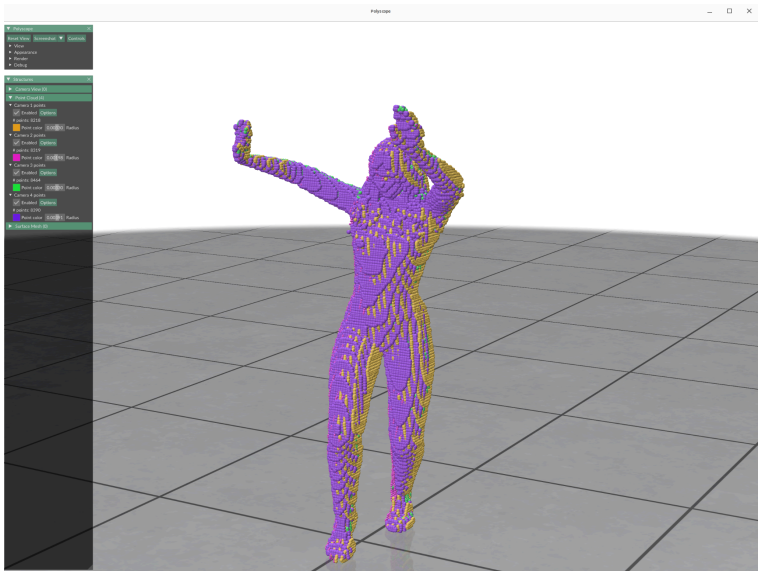


Frame 100:

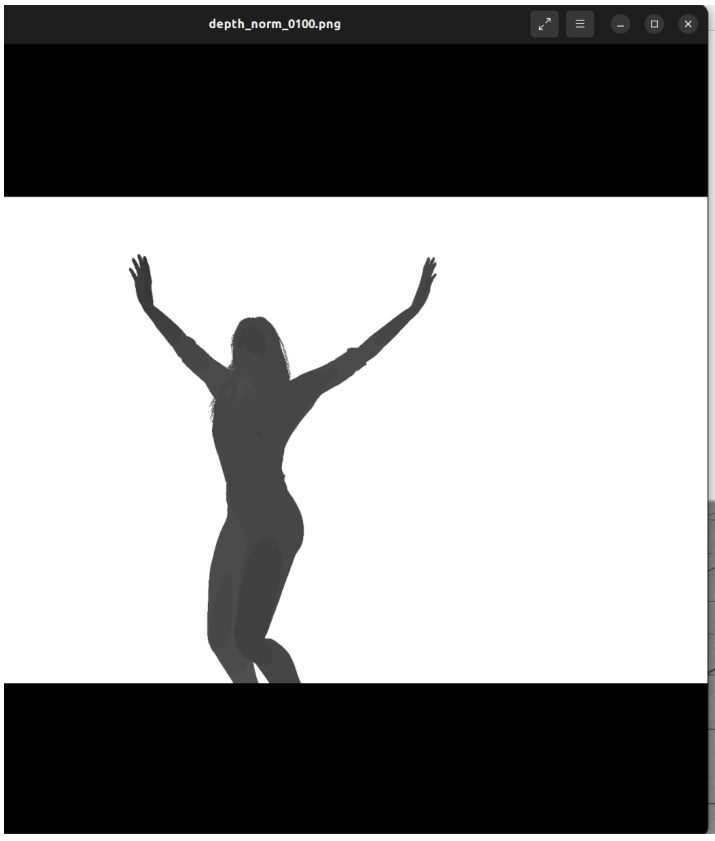


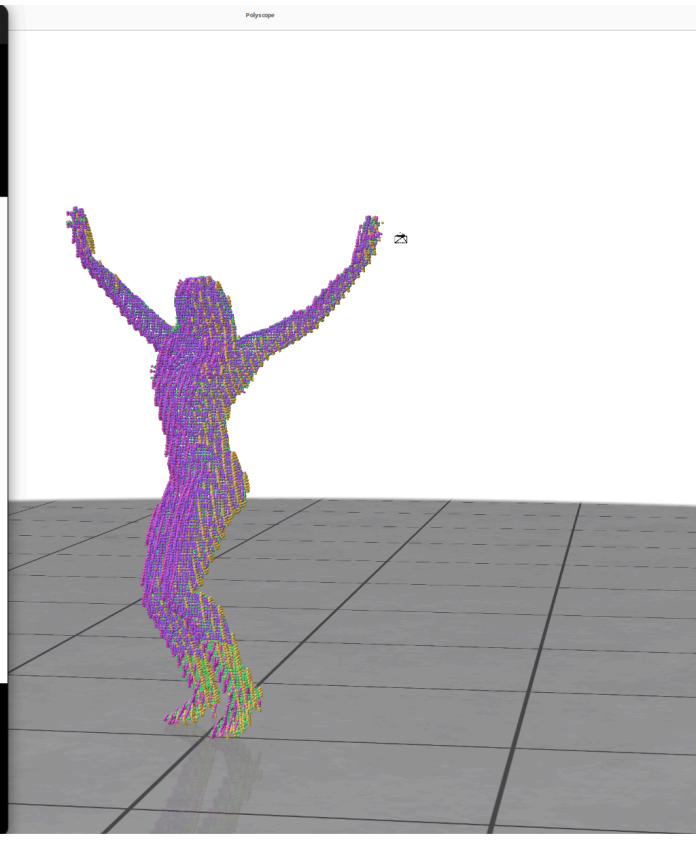
Large Scale: 2.5 - 8.5

Frame 1:



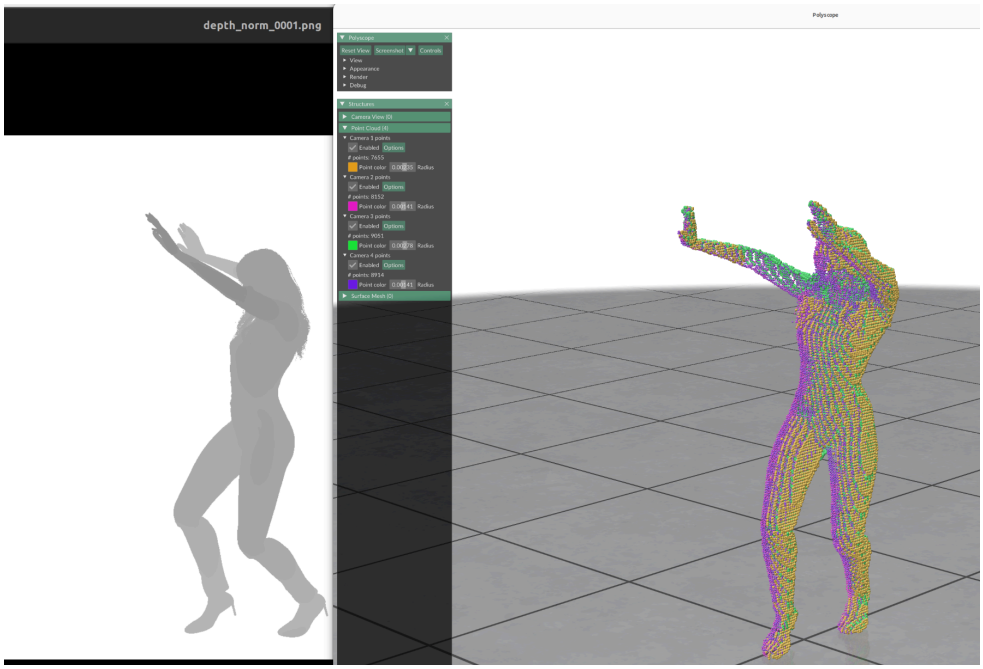
Frame 100:



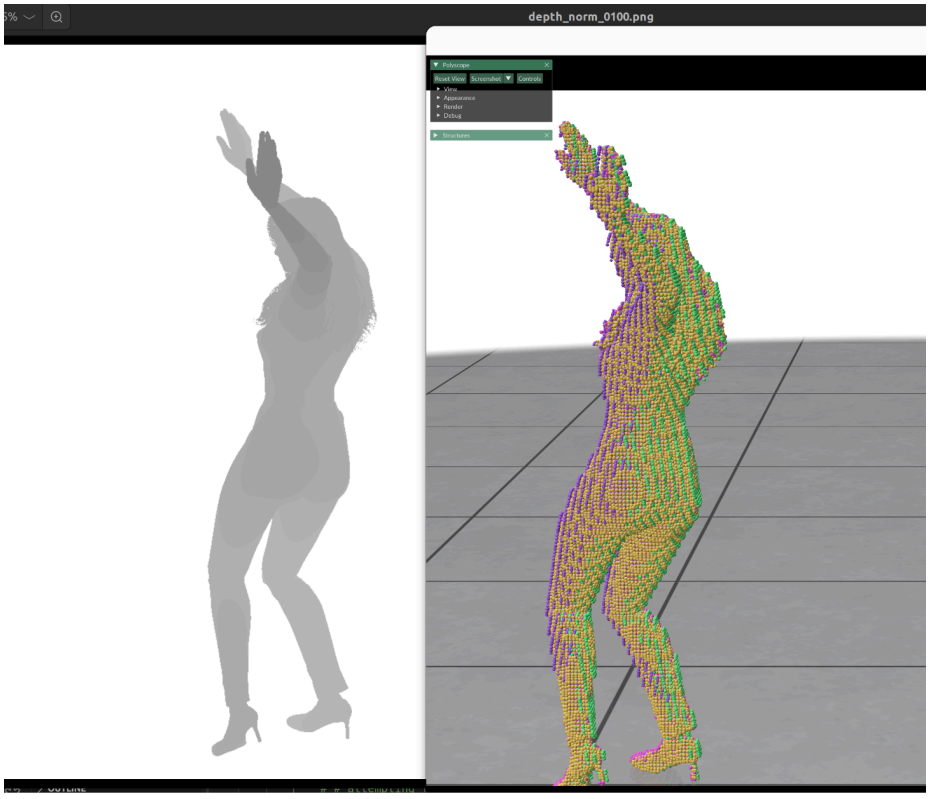


Mid Scale 1: 1.75 - 6.75

Frame 1:

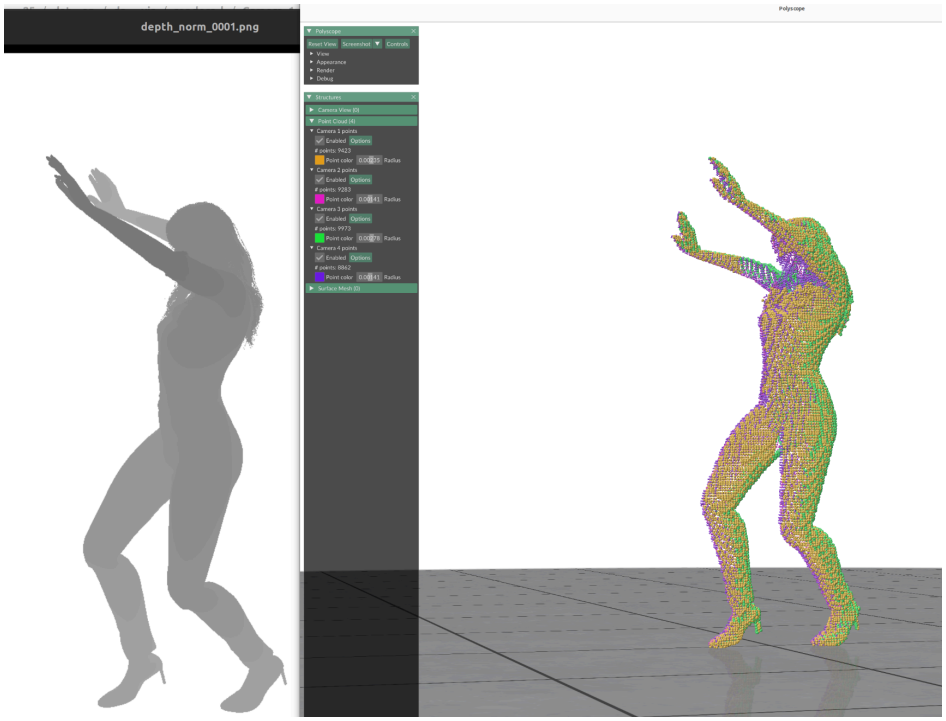


Frame 100:

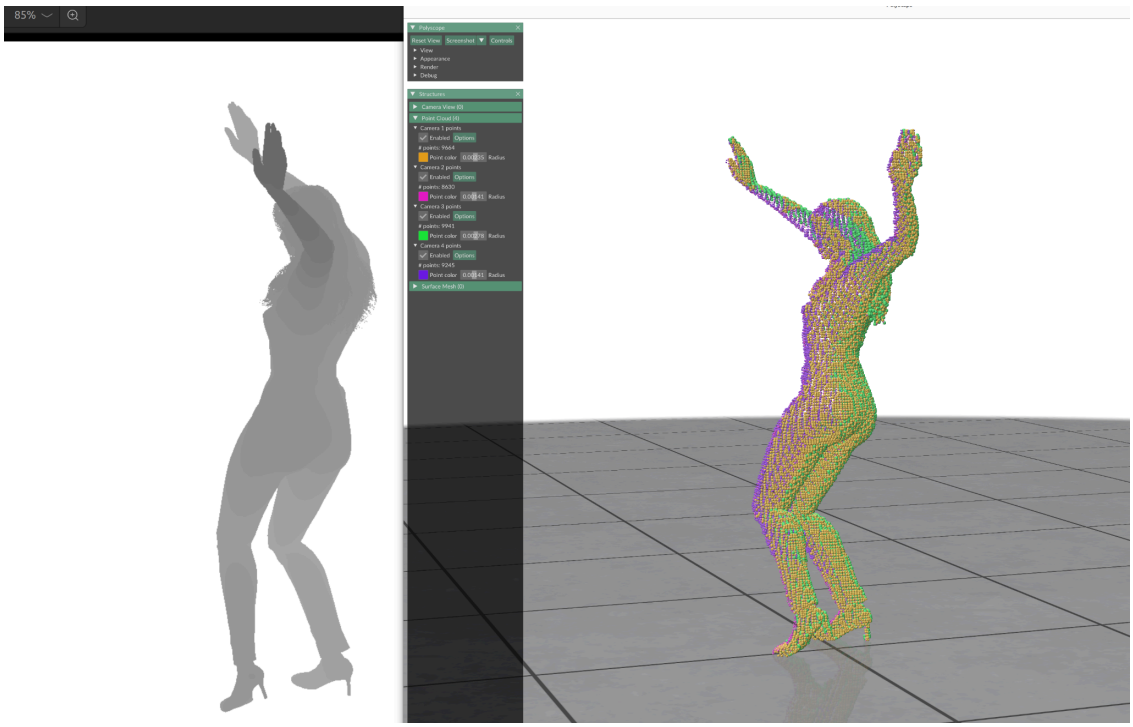


Mid Scale 2:2.75 - 6.75

Frame 1:

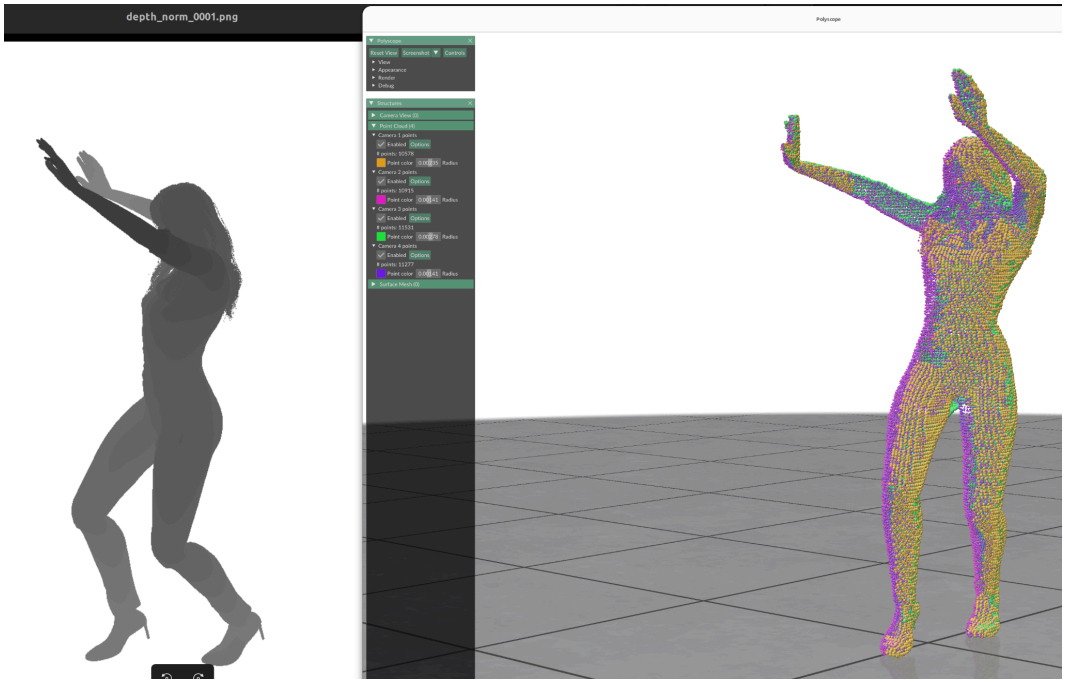


Frame 100:

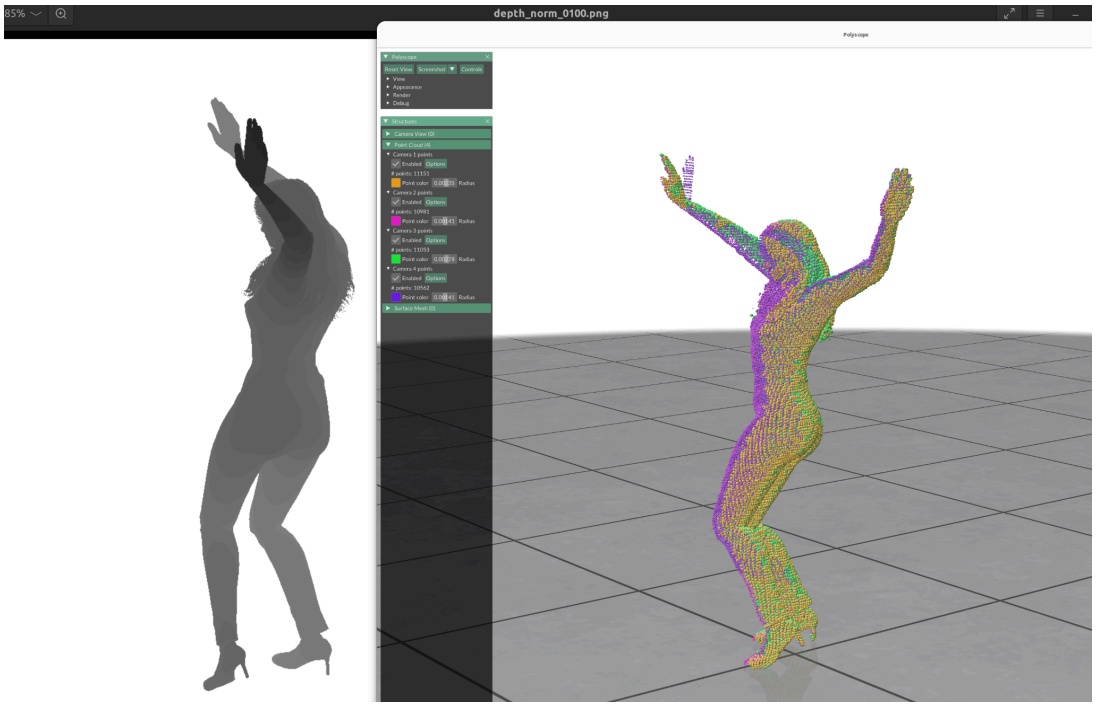


Mid Scale 3: 4 - 6.75

Frame 1:

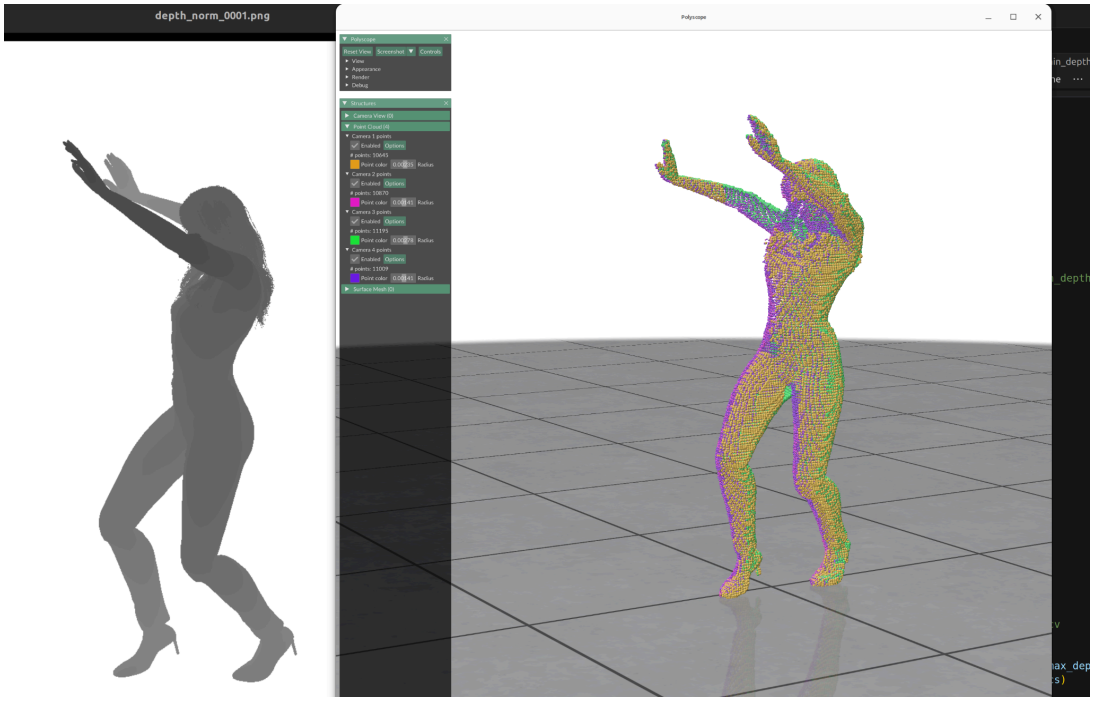


Frame 100:

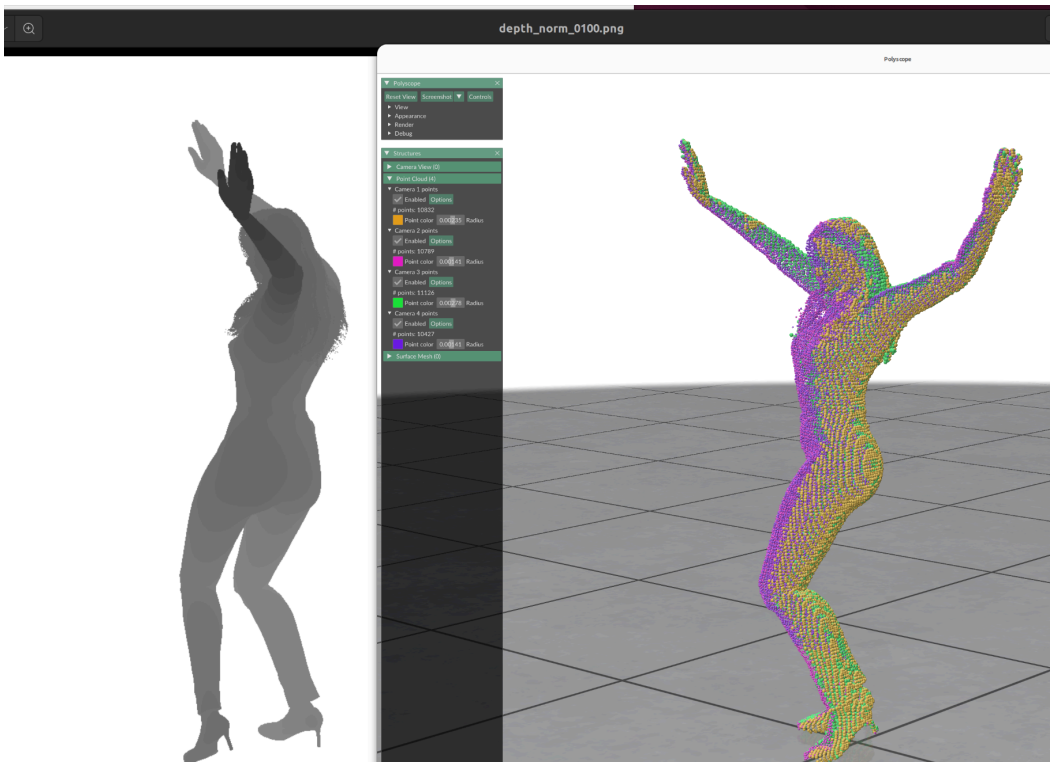


Mid Scale 4: 3.8 - 6.75

Frame 1:

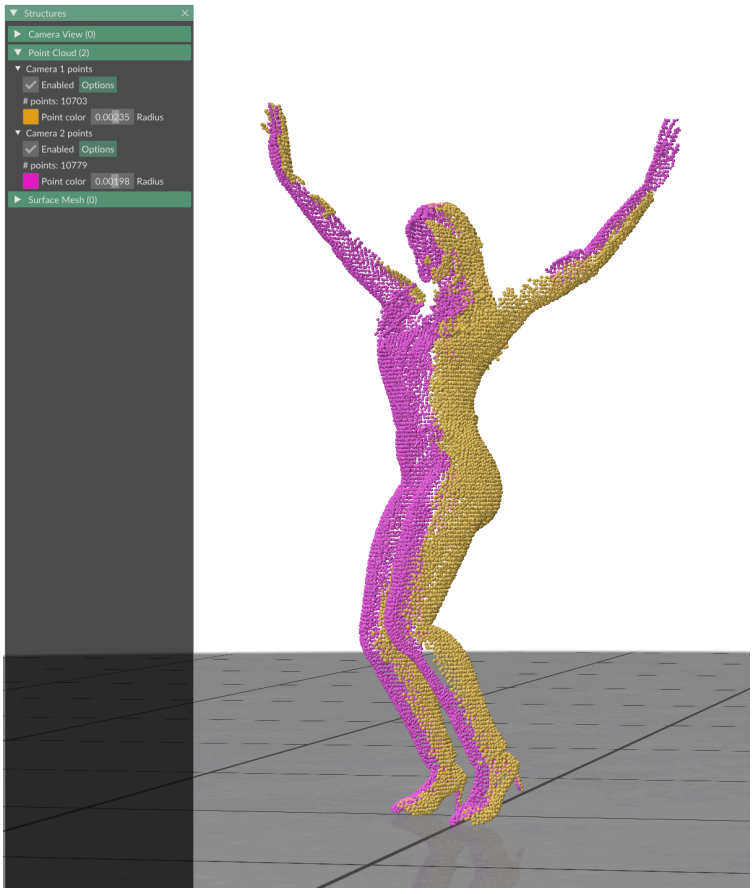


Frame 100:



I think the <u>mid scale 4</u> of 3.8 to 6.75 is the best

But If we want to keep the original 4.5 - 5.5 (or 4 - 6.75 which was close), would we use them as partial meshes?

E.g. 

If so then:

- Frame 7 is when you'd start to get holes from Camera 3
- Frame 33 is when the model would be completely out of view of Camera 3

AND

- Frame 12 is when the model would be beyond the min range of Camera 4 so parts of the legs and arms would show up completely flat
- Frame 30 is when the whole model is beyond the min range of Camera 4 so it would be completely flat by then